

Solution of Mid Term

Divide and Conquer

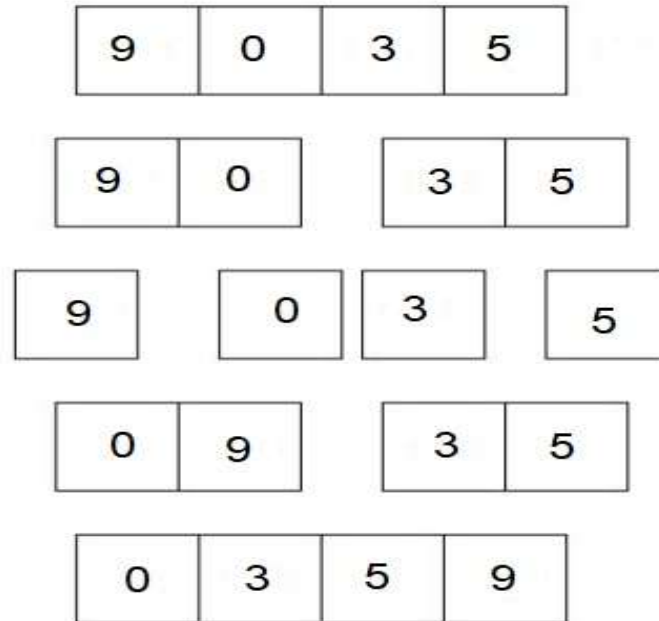
- Given a problem of size n
- Divide into a subproblems
- Each with size $\frac{n}{b}$
- Where $a \geq 1$ and $b > 1$
- Solve each subproblem recursively
- Combine the solutions
- Running time: $T(n) = aT\left(\frac{n}{b}\right) + [merge]$

Merge sort

- **Input:** array $A[1 \dots n]$ of elements
- **Output:** permutation $B[1 \dots n]$ of A such that $B[1] \leq B[2] \leq \dots B[n]$
- Example: $A = [5, 1, 4, 0, 2] \rightarrow B = [0, 1, 2, 4, 5]$
- Applications
 - Organize Files in Folder
 - Contacts in Phone
 - Finding Median
 - Binary Search
 - Data Compression
 - Computer Graphics

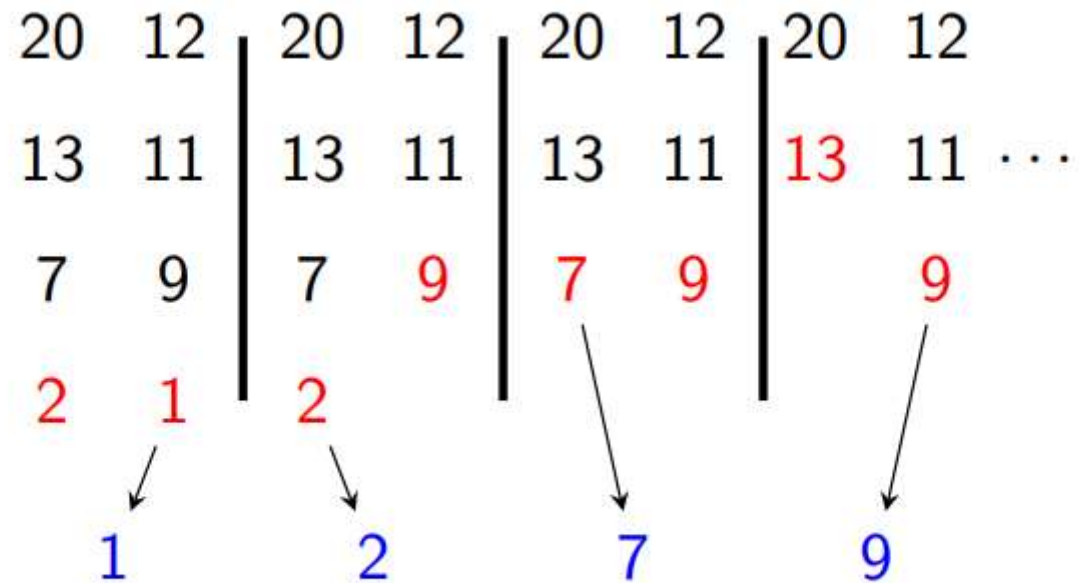
Merge sort

- Divide array into two half
- Continue to divide until we are left with one element → Trivially sorted
- Merge elements in sorted order



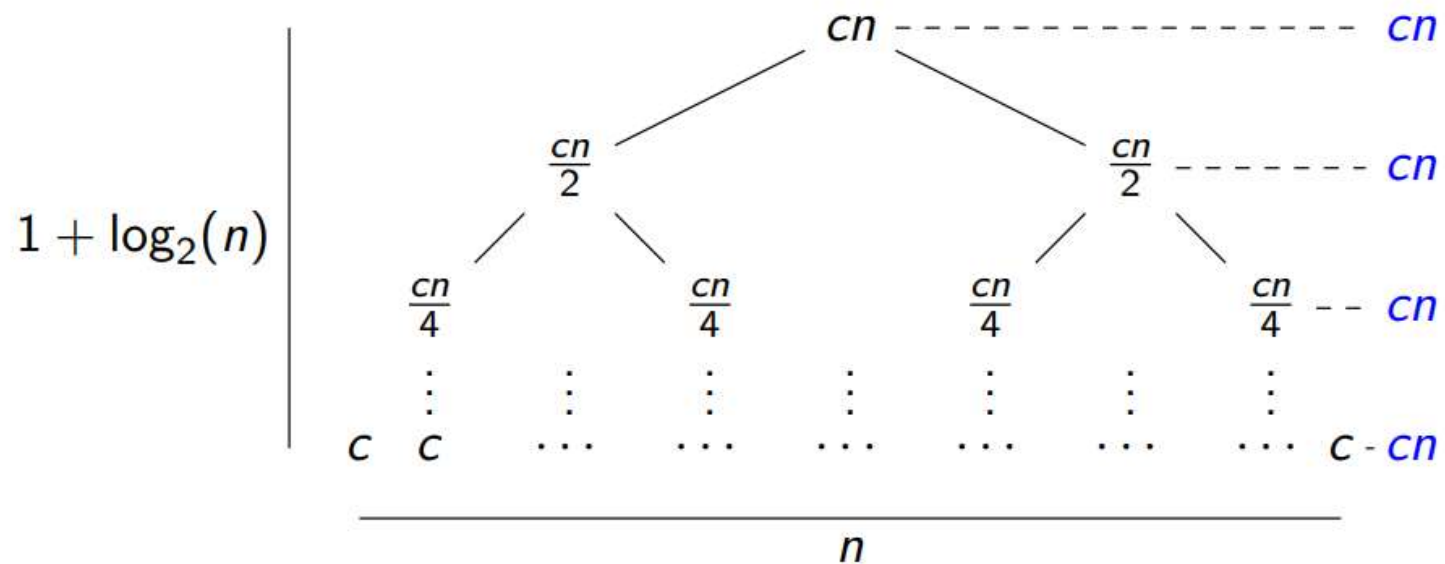
Merge Operation

2 Subarrays to merge: [20,13,7,2] and [12, 11, 9, 1]



Solving Recurrences

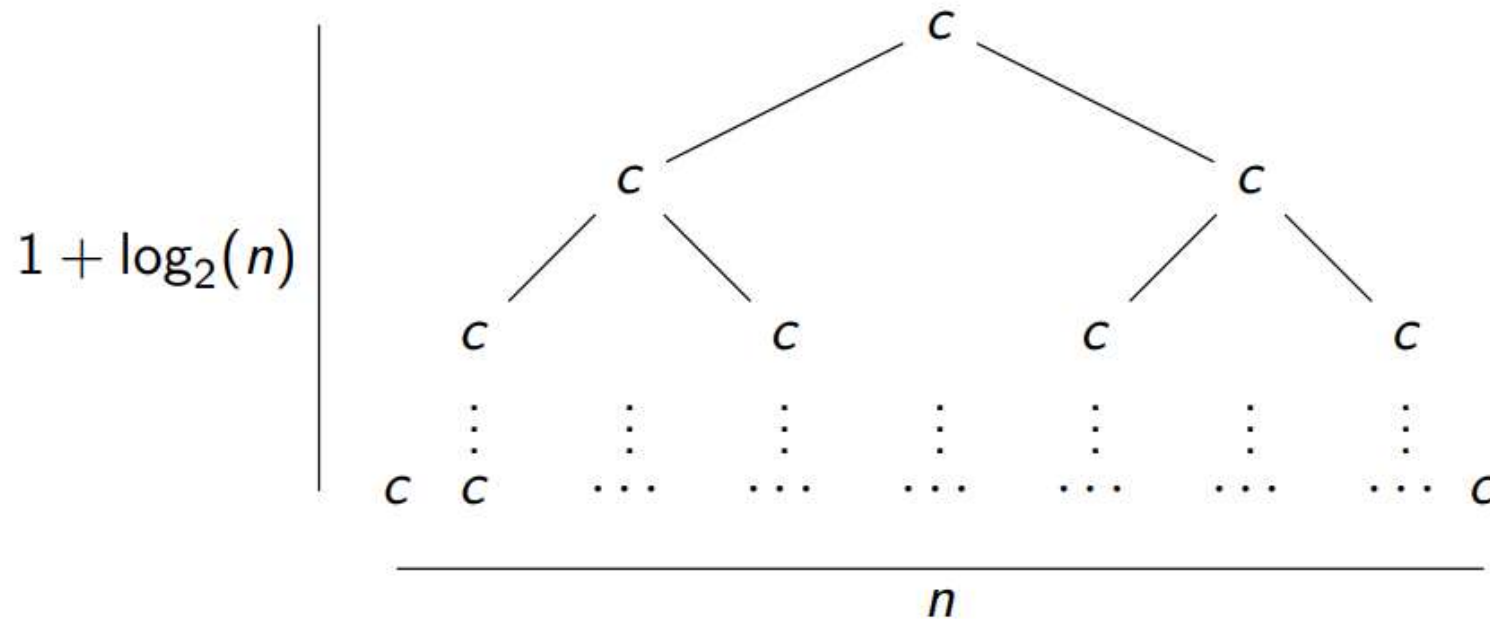
- Solve $T(n) = 2T\left(\frac{n}{2}\right) + cn$, where $c > 0$ is constant
- Recursion Tree



- Equal amount of work done in each level
- Total: $cn \times (1 + \log_2(n)) = cn + cn \log_2(n) = \Theta(n \log_2(n))$

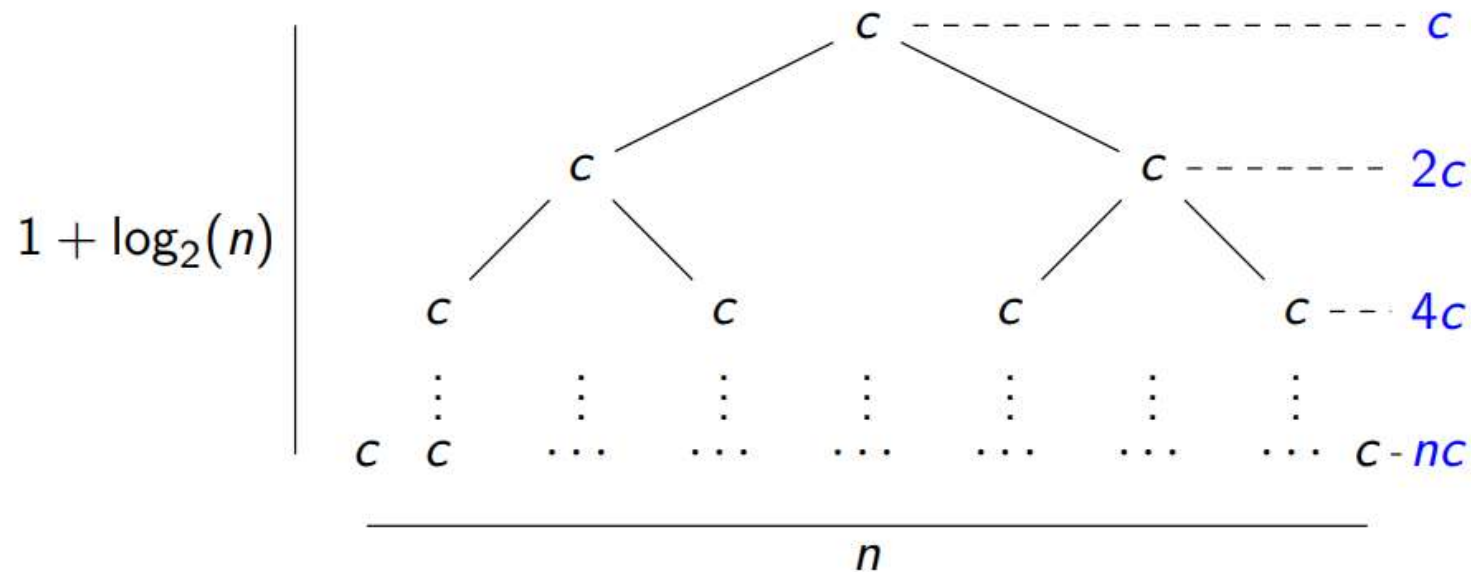
Solving Recurrences

- Solve $T(n) = 2T\left(\frac{n}{2}\right) + c$, where $c > 0$ is constant



Solving Recurrences

- Solve $T(n) = 2T\left(\frac{n}{2}\right) + c$, where $c > 0$ is constant



- Most of the work done in leaves

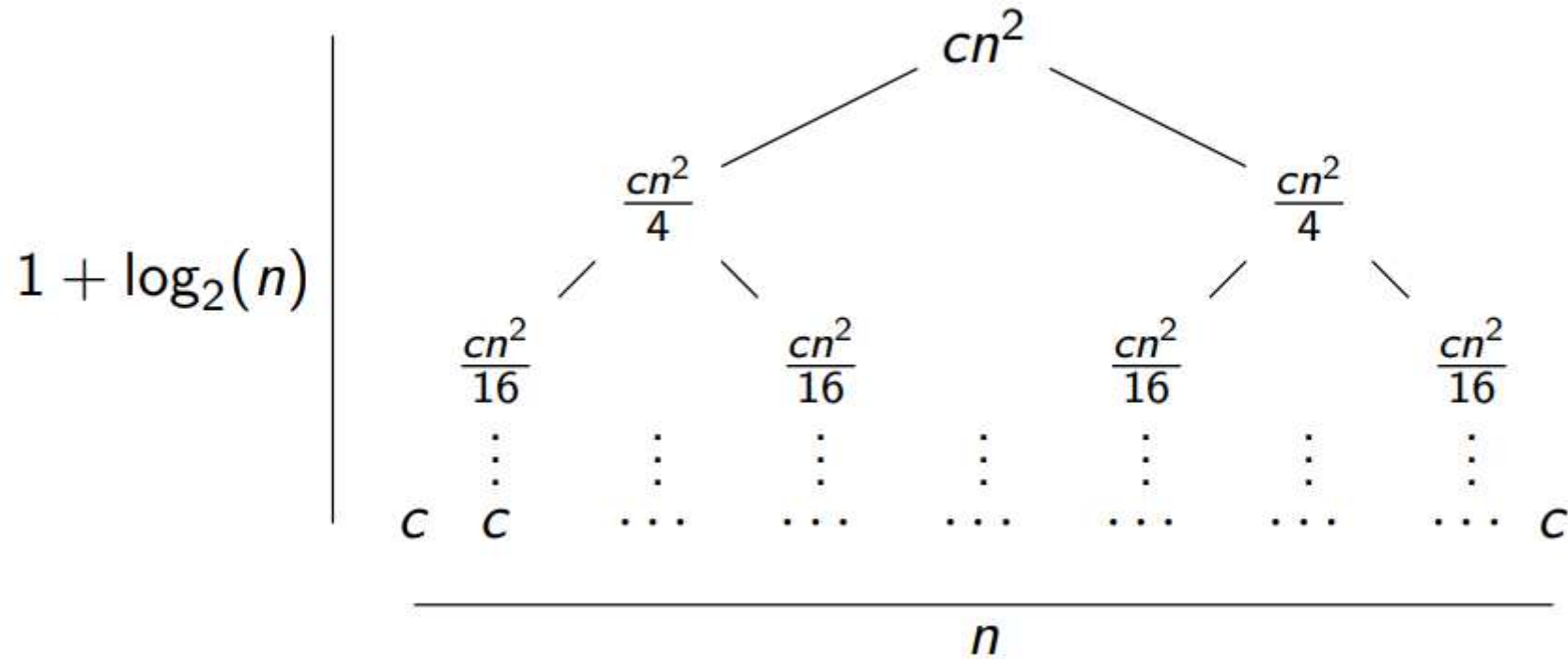
■ $T(n) = nc + \frac{nc}{2} + \frac{nc}{4} + \dots + \frac{nc}{n/2} + \frac{nc}{n}$

$$\Rightarrow T(n) = nc \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n/2} + \frac{1}{n} \right)$$

$$\Rightarrow T(n) = \Theta(n)$$

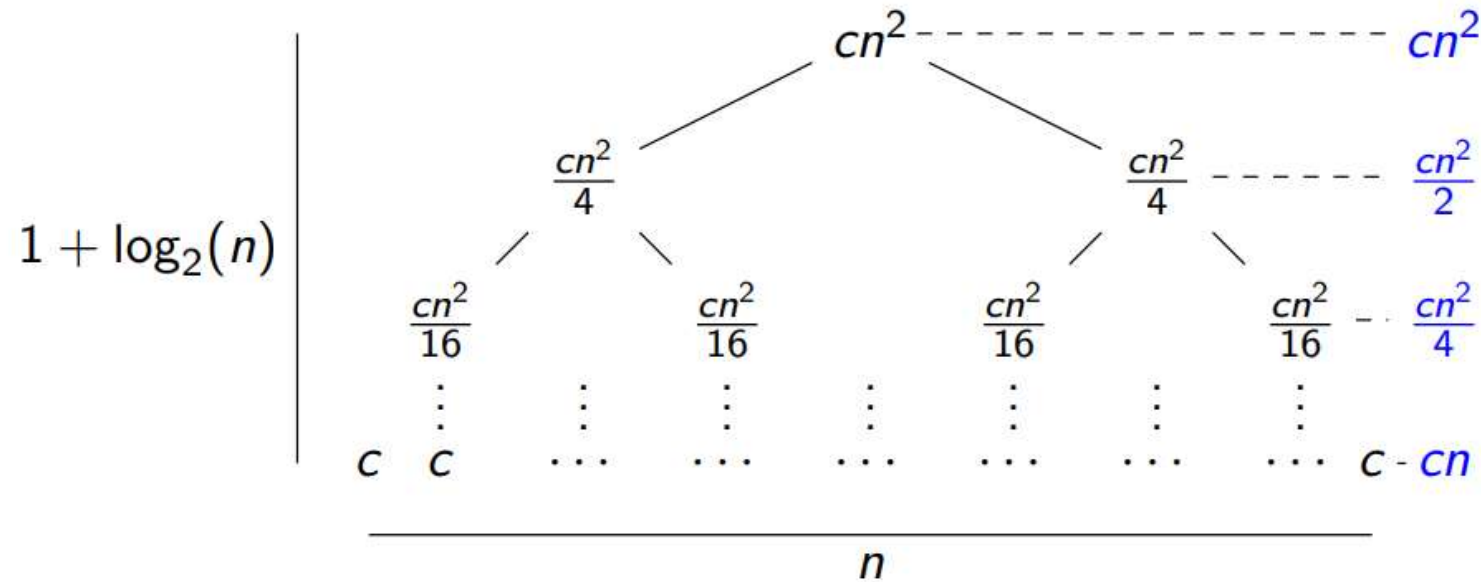
Solving Recurrences

- Solve $T(n) = 2T\left(\frac{n}{2}\right) + cn^2$, where $c > 0$ is constant



Solving Recurrences

- Solve $T(n) = 2T\left(\frac{n}{2}\right) + cn^2$, where $c > 0$ is constant



- Most of the work done in root
- $T(n) = cn^2 + \frac{cn^2}{2} + \frac{cn^2}{4} + \dots + \frac{cn^2}{n}$
- $\Rightarrow T(n) = cn^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n}\right)$
- $\Rightarrow T(n) = \Theta(n^2)$

Solving Recurrences (Master Theorem)

$$T(n) = aT(n/b) + f(n) \text{ where } a \geq 1 \text{ and } b > 1$$

There are following three cases:

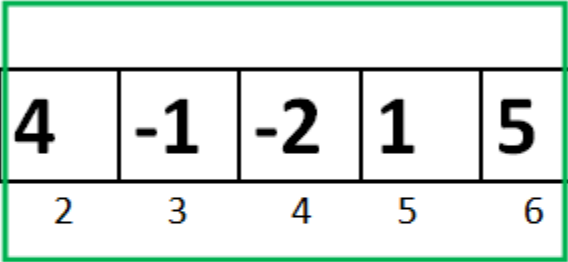
1. If $f(n) = O(n^c)$ where $c < \log_b a$ then $T(n) = \Theta(n^{\log_b a})$
2. If $f(n) = \Theta(n^c)$ where $c = \log_b a$ then $T(n) = \Theta(n^c \log n)$
3. If $f(n) = \Omega(n^c)$ where $c > \log_b a$ then $T(n) = \Theta(f(n))$

Divide and Conquer (Cont...)

Maximum Subarray Sum

Largest Subarray Sum Problem

-2	-3	4	-1	-2	1	5	-3
0	1	2	3	4	5	6	7



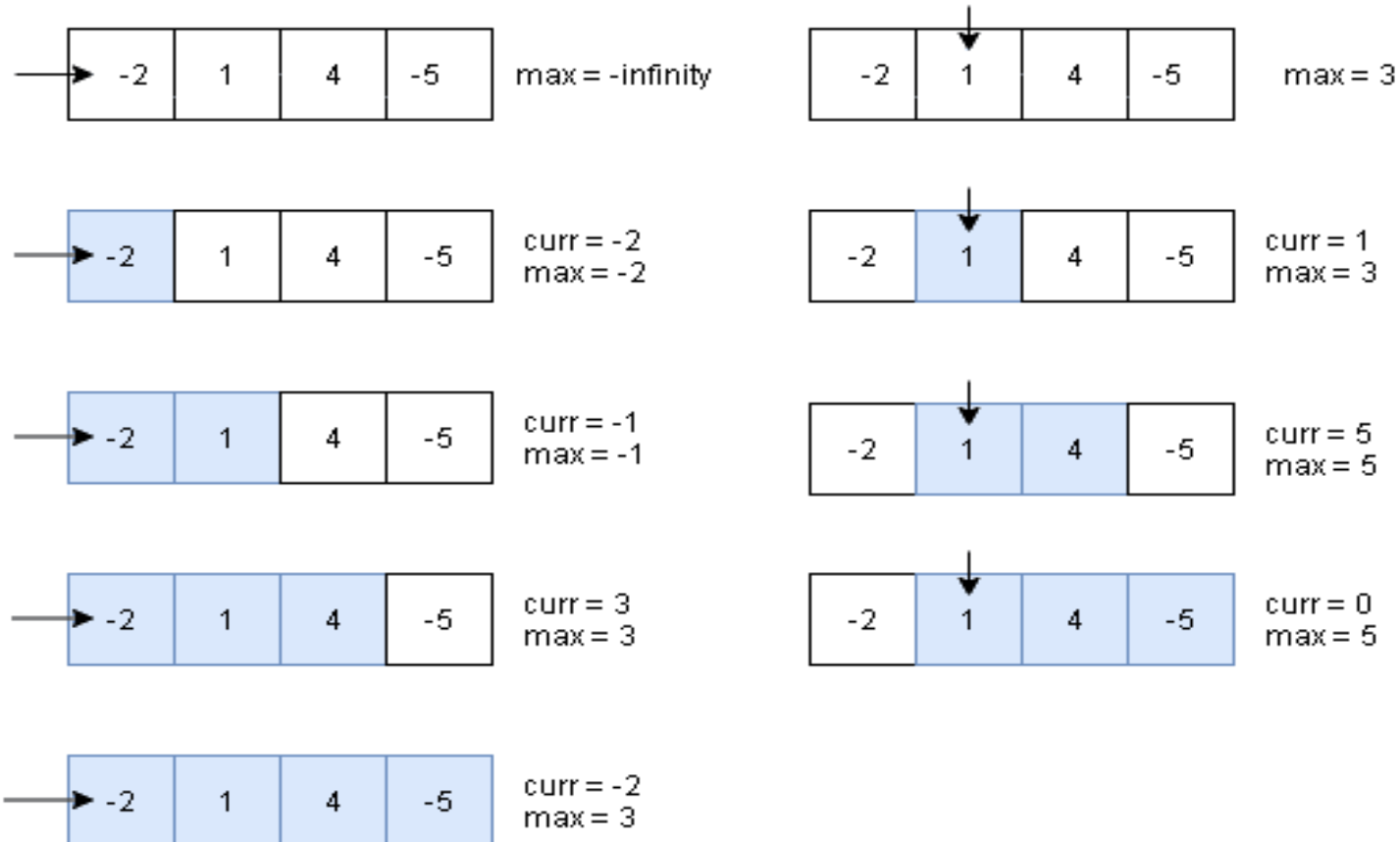
$$4 + (-1) + (-2) + 1 + 5 = 7$$

Maximum Contiguous Array Sum is 7

Maximum Subarray Sum

Array

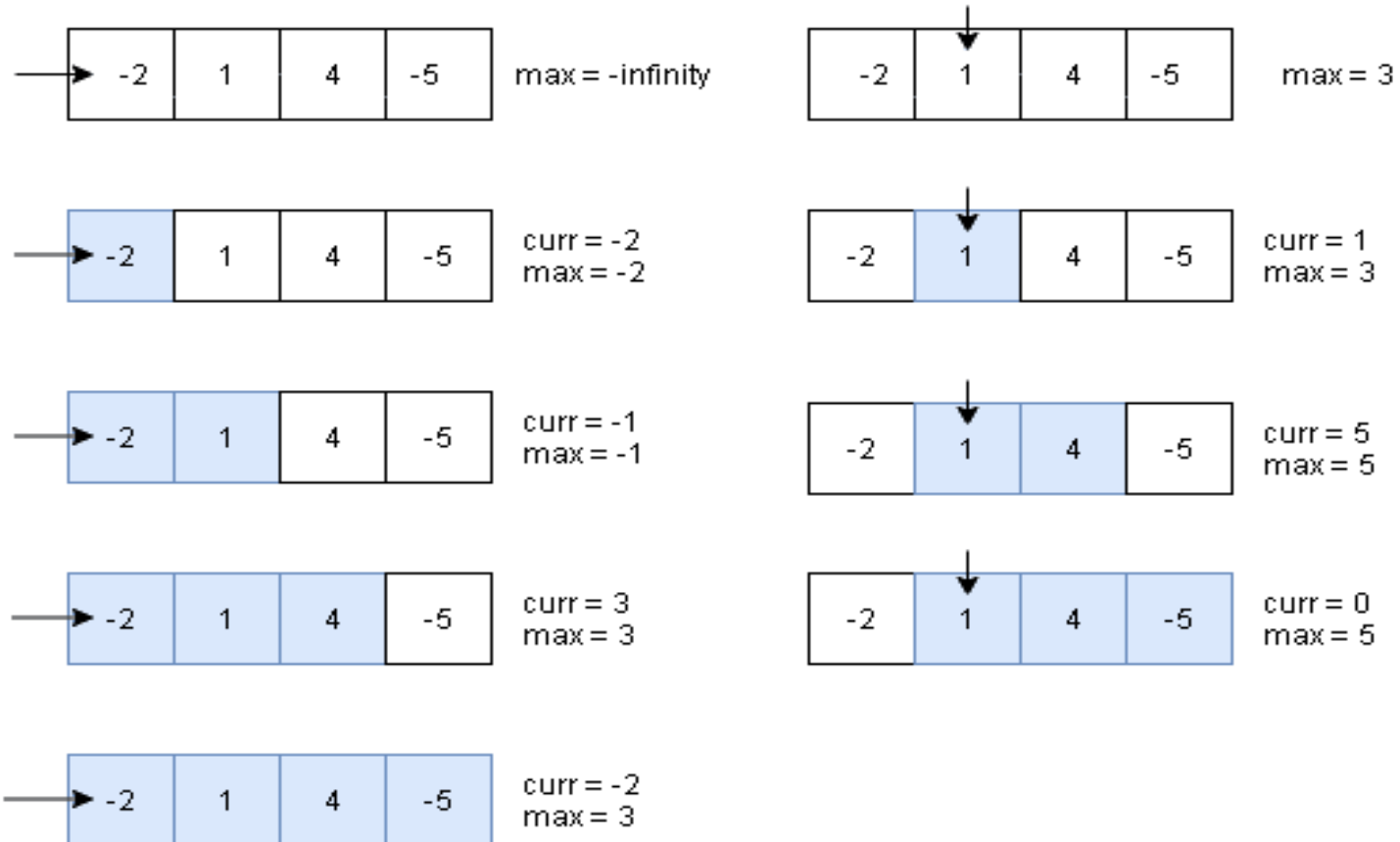
-2	1	4	-5
----	---	---	----



Maximum Subarray Sum

Array

-2	1	4	-5
----	---	---	----



Complexity : $O(N^2)$

Maximum Subarray Sum

- Divide the given array in two halves
- Return the maximum of following three
 - Maximum subarray sum in left half (Make a recursive call)
 - Maximum subarray sum in right half (Make a recursive call)
 - Maximum subarray sum such that the subarray crosses the midpoint
- Time Complexity: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Sum

- Maximum subarray sum such that the subarray crosses the midpoint

```
for (int i = mid; i >= low; i--)
```

```
{
```

```
    sum = sum + arr[i];
```

```
    if (sum > leftpartsum)
```

```
        leftpartsum = sum;
```

```
}
```

```
for (int i = mid+1; i <= high; i++)
```

```
{
```

```
    sum = sum + arr[i];
```

```
    if (sum > rightpartsum)
```

```
        rightpartsum = sum;
```

```
}
```

```
return leftpartsum + rightpartsum
```

Maximum Subarray Sum

- Time Complexity: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
 - $\text{Max}(A[i], A[i] + \text{localmax}[i-1])$

Do we need an array to store localmax?

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
 - $\text{Max}(A[i], A[i] + \text{localmax}[i-1])$

Do we need an array to store localmax?

- Yes, if we don't keep the track of global max
- No, for the previous approach

Maximum Subarray Sum (Kadane's Algorithm)

- Local Max: Max possible value including the i^{th} element.
 - $\text{Max}(A[i], A[i] + \text{localmax}[i-1])$
- Global Max: Update every time localmax is calculated

```
for (i=0; i < n; i++){  
    localmax = max(A[i], A[i] + localmax);  
    if(localmax > globalmax)  
        globalmax = localmax;  
}
```

Maximum Subarray Sum (Kadane's Algorithm)

```
for (i=0; i < n; i++){  
    localmax = max(A[i], A[i] + localmax);  
    if(localmax > globalmax)  
        globalmax = localmax;  
}
```

```
for (i=0; i < n; i++){  
    localmax = A[i] + localmax;  
    if(localmax > globalmax)  
        globalmax = localmax;  
    if(localmax < 0)  
        localmax = 0;  
}
```

Divide and Conquer

Longest Nice Substring

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase

- aAbBAAA – Nice string
- Abc – Not nice
- abBxCDScsedEad - ??

Longest Nice Substring

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase

- aAbB – Nice string
- Abc – Not nice
- abBxCDScsedEad – Not nice (But there are substrings, which are nice)

Longest Nice Substring

- aAAbB – Nice string
- Abc – Not nice
- a**bB**x**CD****Scsed****E**ad – (Goal is to find the longest nice substring)

Trivial Solution

- Find all possible substring
 - Check which one of them are nice
 - Print the one with highest length
-
- Number of all possible nonempty substring: $n(n+1)/2$
 - $O(n^2)$

Solution

- If the string length is less than 2, return empty string.
- Keep the track of occurrence of every alphabets.

(To check whether a letter have its counter-letter in the string or not)

Solution

- If the string length is less than 2, return empty string.
- Keep the track of occurrence of every alphabets.

```
if(s.length() $<$ 2) return "";

set <char> ch_set;
int len = s.length();
for(int i = 0; i<len; i++){
    ch_set.insert(s[i]); // In case of set
                        //ar[ s[i] ] = 1; //In case of array
}
```

Keep the track of occurrence of every alphabets

Using array:

```
ch = s[i]
if ( ar[toupper(ch)] && ar[ tolower(ch)] )
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1		65	66	67	...	88	89	90	...	97	98	99	...	120	121	122

Using set:

```
ch = s[i]
if( ch_set.contains(toupper(ch)) && ch_set.contains(tolower(ch)) )
```

Solution

- If the string length is less than 2, return empty string.
- Keep the track of occurrence of every alphabets.
- Run a loop from the beginning of the string (s)
 - until you found a letter which does not have its counter-letter in the string
 - Or reached to the end of the string
- If it is the end of the string, return s
- Otherwise
- Split the string into two substring from the left and right of the non-matching character and return the longest nice substring from both parts

Solution

- Run a loop up to the end of the string
 - `Check if (ch_set.count(toupper(ch)) && ch_set.count(tolower(ch)) )`
 - `if true, check the next letter.`
 - `else`
 - `Recursively call the left and right substring to find the longest nice substring in them.`
 - `Return the bigger one among them`

Solution

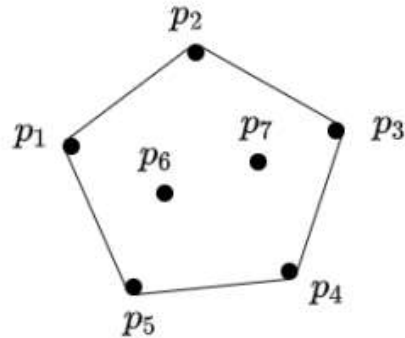
- Recursively call the left and right substring:
 - Left: `s1 = f1(s.substr(0,i))`
 - Right: `s2 = f1(s.substr(i+1, len-i+1))`

- `string longestNiceSubstring(string s) {`
- `if(s.length()<2) return "";`
- `set <char> ch_set;`
- `int len = s.length();`
- `for(int i = 0; i<len; i++){`
- `ch_set.insert(s[i]);`
- `}`
- `for(int i = 0; i<len; i++){`
- `char ch = s[i];`
- `if(ch_set.count(toupper(ch)) && ch_set.count(tolower(ch)))`
- `continue;`
- `string s1 = longestNiceSubstring(s.substr(0,i));`
- `string s2 = longestNiceSubstring(s.substr(i+1));`
- `return s1.length() >= s2.length() ? s1:s2;`
- `}`
- `return s;`
- `}`

Problem Link

- <https://leetcode.com/problems/longest-nice-substring/description/>

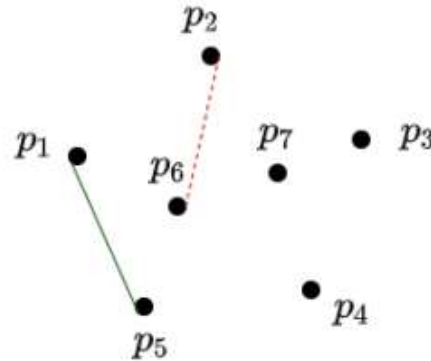
Convex Hull Problem



p_1 p_2 p_3 p_4 p_5

- Given n points in a plane, $S = \{(x_i, y_i) \mid i = 1, 2, \dots, n\}$
 - No two have the same x
 - No two have the same y
 - No three are in a line
- Find the smallest convex polygon containing all points in S , $CH(S)$
 - Sequence of boundary points
 - Clockwise order

Convex Hull Problem

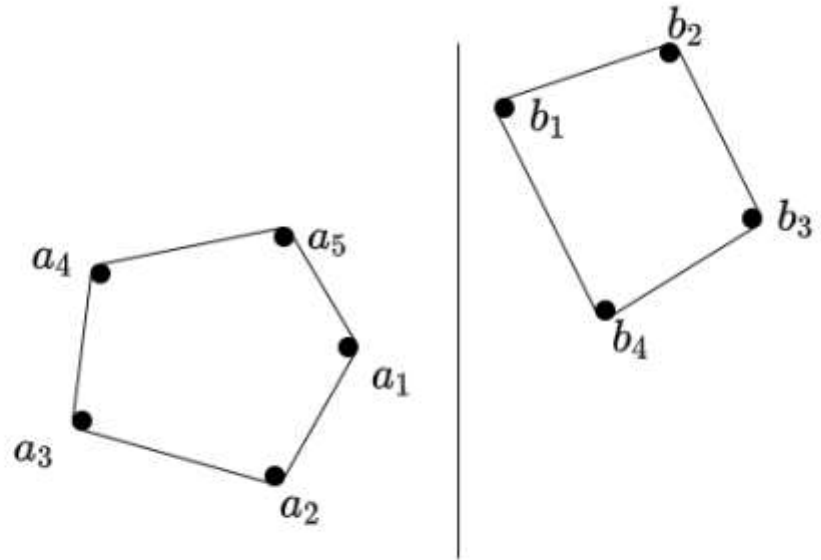


- For each pair of points
- Check if the line segment is on the convex hull or not
 - If rest of the points are on one side $\rightarrow$ Yes
 - If rest of the points are on both sides $\rightarrow$ No
- Complexity
 - Total pair of points: $O(n^2)$

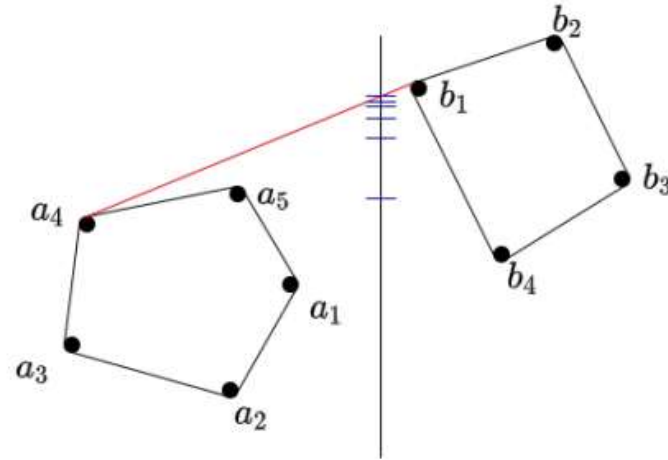
Convex Hull Problem

- Sort the points by x -coordinates
- For input set S ,
 - Divide into left half A and right half B using x -coords
 - Compute $CH(A)$
 - Compute $CH(B)$
 - Combine
- Solve sufficiently smaller problem using brute force

Convex Hull Problem

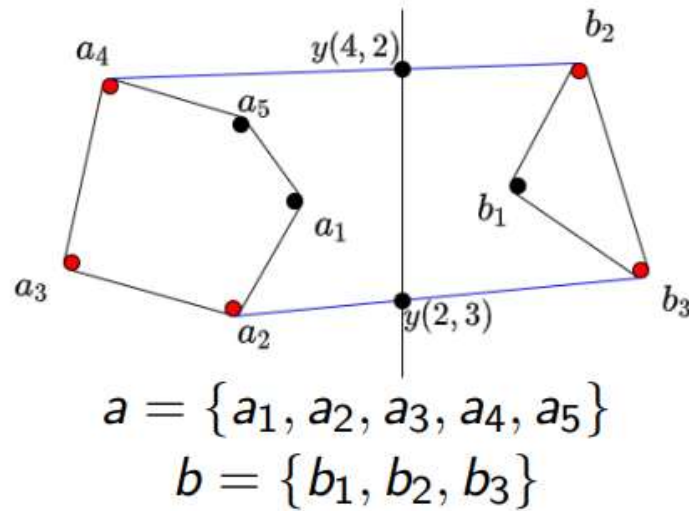


Convex Hull Problem



- Picking the max y for both is not enough
- Need to maximize $y(i, j)$
- Two-Finger Algorithm
 - Start with line segment using rightmost of a and leftmost of b
 - Move clockwise for $b \rightarrow$ Update if it improves $y(i, j)$
 - Move anti clockwise for $a \rightarrow$ Update if it improves $y(i, j)$
 - Repeat until $y(i, j)$ converges
- Similarly find lower tangent

Convex Hull Problem



- Found two tangents? Merge.
- Cut and Paste Method
 - Start from left of upper tangent
 - Go to the right of upper tangent
 - Keep going clockwise until you reach right of lower tangent
 - Move to left of lower tangent
 - Keep going clockwise until you reach left of upper tangent

Convex Hull Problem (Complexity)

- Sort $\rightarrow O(n \log_2(n))$ (e.g. Merge Sort) or $O(n)$ (e.g. Radix Sort)
- Divide and Conquer
 - Divide
 - ▶ Choose partition at the center
 - ▶ $T(n) = 2T\left(\frac{n}{2}\right)$
 - Merge
 - ▶ Two finger algorithm $\rightarrow O(n)$
 - ▶ Merge two lists $\rightarrow O(n)$
 - Total
 - ▶ $T(n) = 2T\left(\frac{n}{2}\right) + O(n) \rightarrow$ Merge sort
 - ▶ $T(n) = O(n \log_2(n))$
- Total Complexity $\rightarrow O(n \log_2(n))$

Matrix Multiplication

- Multiply two matrices A and B

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

- Compute n^2 matrix entries
- Each is sum of n values

- Complexity: $O(n^3)$

Matrix Multiplication

- Assume $n = 2^k$ (Or make it, by appending zeros)
- Divide the matrix in four $\frac{n}{2} \times \frac{n}{2}$ matrices
- Continue dividing until we get single element $\rightarrow$ Use brute force
- Combine

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

- Cost
 - Multiplications of size $\frac{n}{2} \rightarrow 8$
 - Addition Count: $4 \times \frac{n^2}{2}$
 - $T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2) = \Theta(n^3)$

Matrix Multiplication (Strassen's Method)

- 7 multiplication instead of 8

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

- Cost: $T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2) = \Theta(n^{\log_2(7)}) \approx \Theta(n^{2.8074})$